JoshuaThomas_FRA_Extended_Project

March 10, 2026

1 Problem Statement

1.1 Context

Bankruptcy prediction is a crucial component of financial risk management that protects the interests of creditors, investors, and other stakeholders. Predicting a company's impending bankruptcy can help with timely interventions and smart decision-making, which can reduce losses and promote stability in the economy. Predictive modeling can benefit from the abundance of financial data provided by US corporations listed on major exchanges such as the New York Stock Exchange (NYSE) and NASDAQ, which are subject to regulatory scrutiny and strict financial reporting requirements. A firm is considered bankrupt, according to the Securities Exchange Commission (SEC), if it files for bankruptcy under the Bankruptcy Code's Chapter 11 (reorganization) or Chapter 7 (liquidation) provisions.

1.2 Objective

A well-known financial analytics company wants to create a Bankruptcy Prediction Tool to help regulators, investors, and financial institutions assess the bankruptcy risk of US publicly traded corporations. The program will evaluate past financial data using cutting-edge machine learning algorithms to find important signs and trends related to bankruptcy. The following are this tool's main goals:

1. Bankruptcy Risk Assessment: Provide a probabilistic estimate of a company's likelihood of filing for bankruptcy within a specified time frame (e.g., one year), allowing stakeholders to make informed decisions and take preventive measures.
2. Early Warning System: Develop an early warning system that flags companies exhibiting financial distress signals, enabling proactive risk management and strategic planning.
3. Financial Health Analysis: Analyze various financial metrics to offer a comprehensive assessment of a company's financial health, highlighting areas of concern and potential vulnerabilities.

As part of the data science team in the firm, you have been provided with a dataset containing financial metrics of various companies. The task is to analyze the data and develop a predictive model using machine learning techniques to identify whether a given company is at risk of bankruptcy in the near future. The model will help the organization anticipate potential financial distress and enable proactive measures to manage risks effectively.

1.3 Data Dictionary

1. **Company_id**: Unique identifier for each company
2. **Current_assets**: Total current assets (in millions)
3. **Cost_of_goods_sold**: Cost of goods sold (in millions)
4. **Depreciation_and_amortization**: Depreciation and amortization expenses (in millions)
5. **EBITDA**: Earnings Before Interest, Taxes, Depreciation, and Amortization (in millions)
6. **Inventory**: Value of inventory (in millions)
7. **Net_income**: Net income (profit or loss) (in millions)
8. **Total_receivables**: Total receivables (in millions)
9. **Market_value**: Market value of the company (in millions)
10. **Net_sales**: Net sales or revenue (in millions)
11. **Total_assets**: Total assets (in millions)
12. **Total_long_term_debt**: Total long-term debt (in millions)
13. **EBIT**: Earnings Before Interest and Taxes (in millions)
14. **Gross_profit**: Gross profit (in millions)
15. **Total_current_liabilities**: Total current liabilities (in millions)
16. **Retained_earnings**: Retained earnings (in millions)
17. **Total_revenue**: Total revenue (in millions)
18. **Total_liabilities**: Total liabilities (in millions)
19. **Total_operating_expenses**: Total operating expenses (in millions)
20. **Bankrupt**: Bankruptcy status (1 = Bankrupt, 0 = Not Bankrupt)

2 Please read the instructions carefully before starting the project.

This is a commented Python Notebook file in which all the instructions and tasks to be performed are mentioned. * Blanks '_____' are provided in the notebook that needs to be filled with an appropriate code to get the correct result. With every '_____' blank, there is a comment that briefly describes what needs to be filled in the blank space. * Identify the task to be performed correctly, and only then proceed to write the required code. * Fill the code wherever asked by the commented lines like "# write your code here" or "# complete the code". Running incomplete code may throw error. * Please run the codes in a sequential manner from the beginning to avoid any unnecessary errors. * Add the results/observations (wherever mentioned) derived from the analysis in the presentation and submit the same.

3 Importing necessary libraries

[1]:
```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
```

```
import statsmodels.api as SM
from sklearn import metrics

from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_curve,
    roc_auc_score
)

from sklearn.ensemble import RandomForestClassifier

import warnings
warnings.filterwarnings('ignore')
```

4 Loading the Data

[2]:
```
# uncomment and run the following lines for Google Colab
# from google.colab import drive
# drive.mount('/content/drive')
```

[3]:
```
df = pd.read_csv('C:/Users/Cifer/OneDrive/Desktop/My Great Learning/Finance and␣
↪Risk Analytics   11/Extended Project/Bankruptcy_Data.csv')  ##  Fill the␣
↪blank to read the data
df.head()
```

[3]:
```
  Company_id  Current_assets  Cost_of_goods_sold  \
0        C_3           6.494              15.700
1        C_6        6540.000           19419.000
2        C_8         189.471             125.059
3       C_12          34.617             104.017
4       C_14          90.558             148.628

   Depreciation_and_amortization   EBITDA  Inventory  Net_income  \
0                          0.761    0.381      3.488      -1.738
1                       1404.000 -456.000    822.000   -1762.000
2                         11.600   44.563     48.423      21.354
3                          3.165    5.766     13.184       0.812
4                          0.909    7.759     37.818       4.245

   Total_receivables  Market_value  Net_sales  Total_assets  \
0              2.677        2.9667     24.051         8.635
```

```
1           1414.000     3445.0155  18963.000     32841.000
2             49.409      626.5350    232.808       310.252
3             11.964       15.1588    117.326        90.941
4             41.500       65.1701    178.154       105.173

   Total_long_term_debt      EBIT  Gross_profit  Total_current_liabilities  \
0                 3.873    -0.380         8.351                      2.045
1              9834.000 -1860.000      -456.000                   7512.000
2                11.428    32.963       107.749                     37.097
3                35.163     2.601        13.309                     24.589
4                 0.671     6.850        29.526                     35.299

   Retained_earnings  Total_revenue  Total_liabilities  \
0             -8.922         24.051              5.918
1           4042.000      18963.000          27468.000
2             29.890        232.808             55.131
3             18.274        117.326             68.162
4             37.242        178.154             35.970

   Total_operating_expenses  Bankrupt
0                    23.670         0
1                 19419.000         1
2                   188.245         0
3                   111.560         0
4                   170.395         1
```

5 Data Overview

[4]:
```
df.head() ##  Complete the code to view top 5 rows of the data
```

[4]:
```
  Company_id  Current_assets  Cost_of_goods_sold  \
0        C_3           6.494              15.700
1        C_6        6540.000           19419.000
2        C_8         189.471             125.059
3       C_12          34.617             104.017
4       C_14          90.558             148.628

   Depreciation_and_amortization   EBITDA  Inventory  Net_income  \
0                          0.761    0.381      3.488      -1.738
1                       1404.000 -456.000    822.000   -1762.000
2                         11.600   44.563     48.423      21.354
3                          3.165    5.766     13.184       0.812
4                          0.909    7.759     37.818       4.245

   Total_receivables  Market_value  Net_sales  Total_assets  \
0              2.677        2.9667     24.051         8.635
```

```
1           1414.000     3445.0155  18963.000     32841.000
2             49.409      626.5350    232.808       310.252
3             11.964       15.1588    117.326        90.941
4             41.500       65.1701    178.154       105.173

   Total_long_term_debt      EBIT  Gross_profit  Total_current_liabilities  \
0                 3.873    -0.380         8.351                      2.045
1              9834.000 -1860.000      -456.000                   7512.000
2                11.428    32.963       107.749                     37.097
3                35.163     2.601        13.309                     24.589
4                 0.671     6.850        29.526                     35.299

   Retained_earnings  Total_revenue  Total_liabilities  \
0             -8.922         24.051              5.918
1           4042.000      18963.000          27468.000
2             29.890        232.808             55.131
3             18.274        117.326             68.162
4             37.242        178.154             35.970

   Total_operating_expenses  Bankrupt
0                    23.670         0
1                 19419.000         1
2                   188.245         0
3                   111.560         0
4                   170.395         1
```

[5]:
```
df.tail() ##  Complete the code to view last 5 rows of the data
```

[5]:
```
     Company_id  Current_assets  Cost_of_goods_sold  \
1978     C_7172           0.042               0.062
1979     C_7176           6.385              11.054
1980     C_7180        1135.848            1370.382
1981     C_7186           9.453               7.894
1982     C_7288           2.753               5.436

      Depreciation_and_amortization   EBITDA  Inventory  Net_income  \
1978                          0.000   -0.048      0.000      -0.047
1979                          0.201    0.421      0.000       0.175
1980                         71.231  187.991    982.000      11.442
1981                          0.467   -1.672      0.880      -3.047
1982                          0.679   -0.610      1.419      -5.163

      Total_receivables  Market_value  Net_sales  Total_assets  \
1978              0.000        1.0813      0.062         0.046
1979              3.678       23.9363     13.788         6.708
1980             93.704     1626.4270   2517.639      1950.615
1981              1.049       15.6110      6.222        16.247
```

```
1982              1.073        5.9271      8.627         4.072

      Total_long_term_debt     EBIT  Gross_profit  Total_current_liabilities  \
1978                 0.000   -0.048         0.000                      0.063
1979                 0.020    0.220         2.734                      3.136
1980               932.022  116.760      1147.257                    693.749
1981                 1.500   -2.139        -1.672                      3.237
1982                 3.234   -1.289         3.191                      3.144

      Retained_earnings  Total_revenue  Total_liabilities  \
1978             -0.051          0.062              0.063
1979              1.714         13.788              3.156
1980           -205.294       2517.639           1662.044
1981             -9.012          6.222              4.737
1982            -24.875          8.627              6.378

      Total_operating_expenses  Bankrupt
1978                     0.110         0
1979                    13.367         0
1980                  2329.648         0
1981                     7.894         0
1982                     9.237         0
```

[6]:
```
df.shape ##  Complete the code to view dimensions of the data
```

[6]:
```
(1983, 20)
```

- There are 1983 rows and 20 columns in the dataset.

[7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1983 entries, 0 to 1982
Data columns (total 20 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   Company_id                    1983 non-null   object
 1   Current_assets                1983 non-null   float64
 2   Cost_of_goods_sold            1983 non-null   float64
 3   Depreciation_and_amortization 1983 non-null   float64
 4   EBITDA                        1983 non-null   float64
 5   Inventory                     1983 non-null   float64
 6   Net_income                    1983 non-null   float64
 7   Total_receivables             1983 non-null   float64
 8   Market_value                  1983 non-null   float64
 9   Net_sales                     1983 non-null   float64
 10  Total_assets                  1983 non-null   float64
 11  Total_long_term_debt          1983 non-null   float64
```

```
 12  EBIT                          1983 non-null   float64
 13  Gross_profit                  1983 non-null   float64
 14  Total_current_liabilities     1983 non-null   float64
 15  Retained_earnings             1983 non-null   float64
 16  Total_revenue                 1983 non-null   float64
 17  Total_liabilities             1983 non-null   float64
 18  Total_operating_expenses      1983 non-null   float64
 19  Bankrupt                      1983 non-null   int64
dtypes: float64(18), int64(1), object(1)
memory usage: 310.0+ KB
```

- The dataset contains 1983 rows and 20 columns.
- 18 columns are float64 type- Numerical data.
- 1 column is int64- (Bankrupt).
- 1 column is of type object- (Company_id).

[8]:
```
# checking for duplicate values
df.duplicated() ##  Complete the code to check duplicate entries in the data
```

[8]:
```
0       False
1       False
2       False
3       False
4       False
        …
1978    False
1979    False
1980    False
1981    False
1982    False
Length: 1983, dtype: bool
```

- There are no duplicate entries.

[18]:
```
#checking for no. of unique values
df.nunique() ##  Complete the code to check unique entries in the data
```

[18]:
```
Current_assets                  1966
Cost_of_goods_sold              1969
Depreciation_and_amortization   1850
EBITDA                          1967
Inventory                       1452
Net_income                      1949
Total_receivables               1847
Market_value                    1982
Net_sales                       1964
Total_assets                    1971
Total_long_term_debt            1452
EBIT                            1963
```

```
Gross_profit                      1968
Total_current_liabilities         1960
Retained_earnings                 1974
Total_revenue                     1964
Total_liabilities                 1962
Total_operating_expenses          1973
Bankrupt                             2
dtype: int64
```

- All columns except Bankrupt contain a significant number of outliers, indicating that the data distribution for many features is skewed or has extreme values.

[10]:
```
df.describe().T
```

[10]:
```
                                 count         mean          std         min  \
Current_assets                  1983.0   485.485038  1790.167597      0.0020
Cost_of_goods_sold              1983.0  1024.313261  4316.517260      0.0000
Depreciation_and_amortization   1983.0    86.553624   400.228725      0.0000
EBITDA                          1983.0   162.429048   880.011401  -5743.0000
Inventory                       1983.0   119.278526   506.207801      0.0000
Net_income                      1983.0   -53.004835  1536.719467 -56121.9000
Total_receivables               1983.0   171.701946   882.459285      0.0000
Market_value                    1983.0  1630.979822  8159.299586      0.0384
Net_sales                       1983.0  1426.305833  5627.381670      0.0020
Total_assets                    1983.0  1641.765901  6932.946945      0.0040
Total_long_term_debt            1983.0   422.770059  1902.188540      0.0000
EBIT                            1983.0    75.875424   697.454528 -10537.0000
Gross_profit                    1983.0   401.992572  1763.762767  -1317.0000
Total_current_liabilities       1983.0   364.712545  1513.951417      0.0040
Retained_earnings               1983.0   133.583326  2235.727309 -57158.0000
Total_revenue                   1983.0  1426.305833  5627.381670      0.0020
Total_liabilities               1983.0  1030.597176  4515.317860      0.0040
Total_operating_expenses        1983.0  1263.876785  4970.260681      0.0670
Bankrupt                        1983.0     0.208775     0.406535      0.0000

                                    25%       50%       75%          max
Current_assets                  14.1920   63.2820  234.6860   36105.0000
Cost_of_goods_sold              14.8690   72.3840  338.1010   76809.0000
Depreciation_and_amortization    1.4920    6.4380   30.2085    9338.0000
EBITDA                          -2.5165    5.3120   55.9780   18632.0000
Inventory                        0.0000    5.7020   44.2140    8923.0000
Net_income                     -14.9175   -0.7970    9.5450    8560.0000
Total_receivables                2.4410   13.8250   61.8435   28813.0000
Market_value                    18.2261   96.3835  588.2171  180090.4065
Net_sales                       23.9920  115.0030  535.4695   97863.0000
Total_assets                    29.4450  131.4280  574.6155  165282.0000
Total_long_term_debt             0.0415    5.6060  125.9820   45247.0000
EBIT                            -7.7785    0.9800   28.2255   16295.0000
```

```
Gross_profit                    6.1790   35.9990  174.9465   41000.0000
Total_current_liabilities       7.6135   27.4420  114.9555   25427.0000
Retained_earnings             -56.0090   -3.8670   64.1825   33896.0000
Total_revenue                  23.9920  115.0030  535.4695   97863.0000
Total_liabilities              11.0270   49.1320  315.2695  110042.0000
Total_operating_expenses       31.2950  111.3780  488.9880   81832.0000
Bankrupt                        0.0000    0.0000    0.0000       1.0000
```

[11]:
```
df.drop(['Company_id'], axis = 1, inplace = True)
```

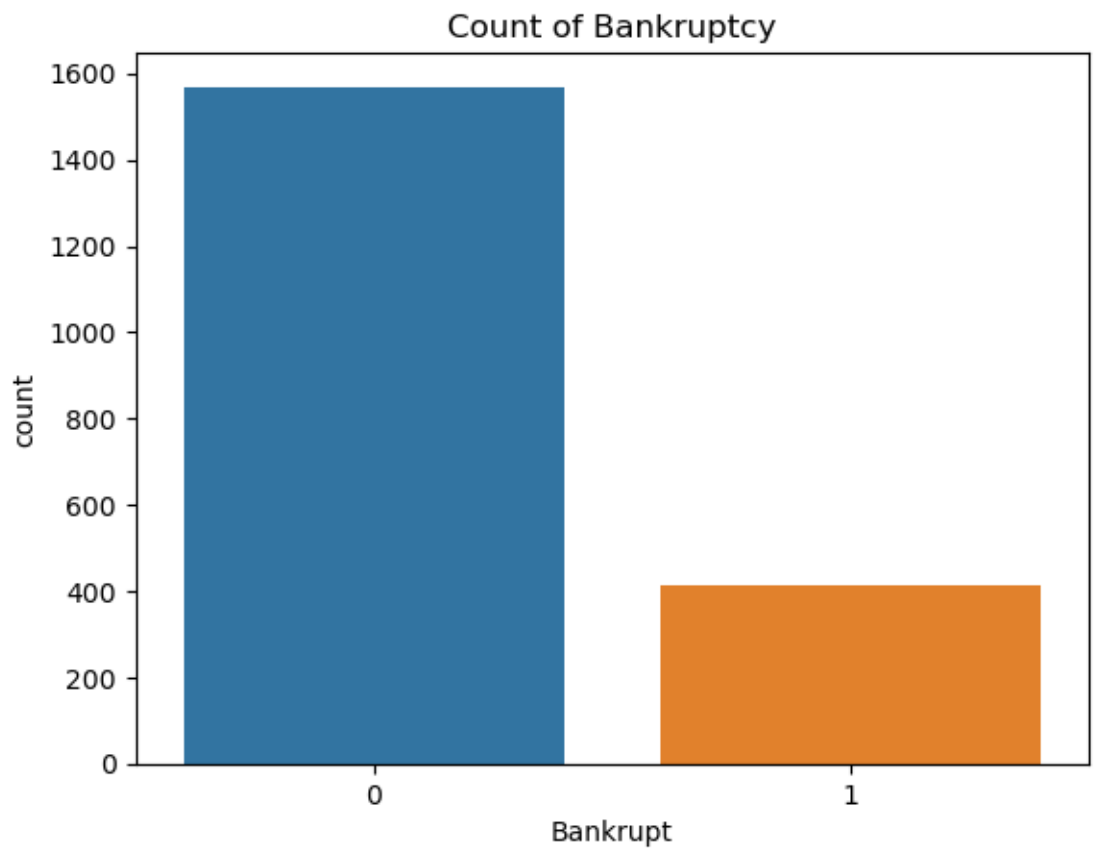
6 Exploratory Data Analysis

6.1 Univariate Analysis

[12]:
```
df["Bankrupt"].unique()   ## Complete the code to check unique values in the␣
↪mentioned column
```

[12]:
```
array([0, 1], dtype=int64)
```

[15]:
```
#Plotting a countplot for the target variable
sns.countplot(x = "Bankrupt", data = df, hue = None)   ## Complete the code to␣
↪get a countplot of the mentionedd column.
plt.title('Count of Bankruptcy')
plt.show()
```

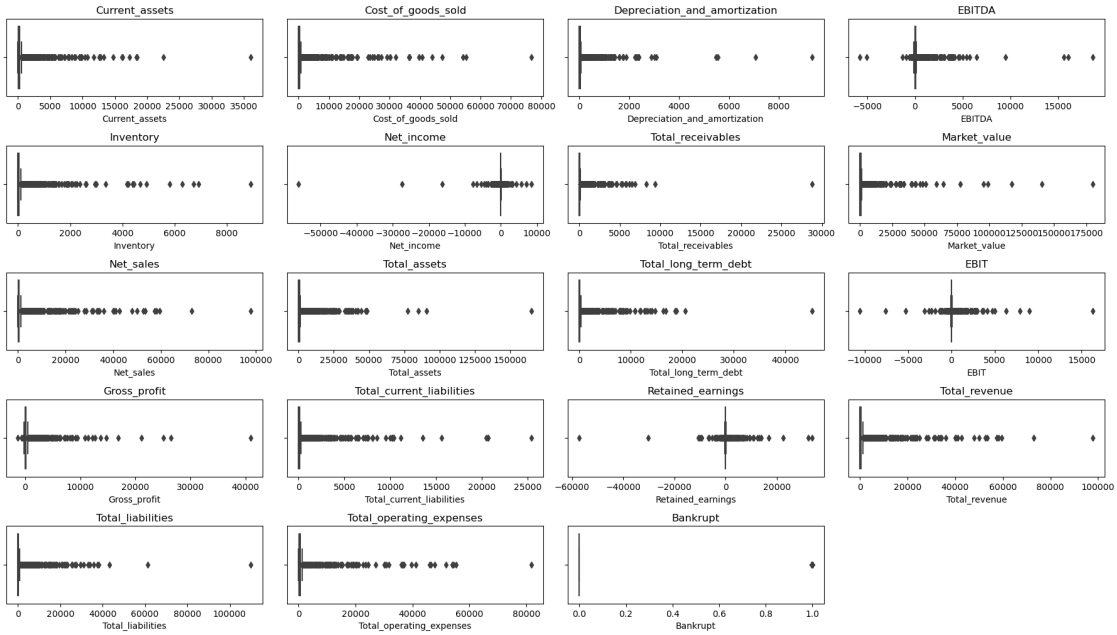


[16]:
```
#Percentage of defaulters
(df.Bankrupt.sum()/len(df)) * 100
```

[16]: 20.87745839636914

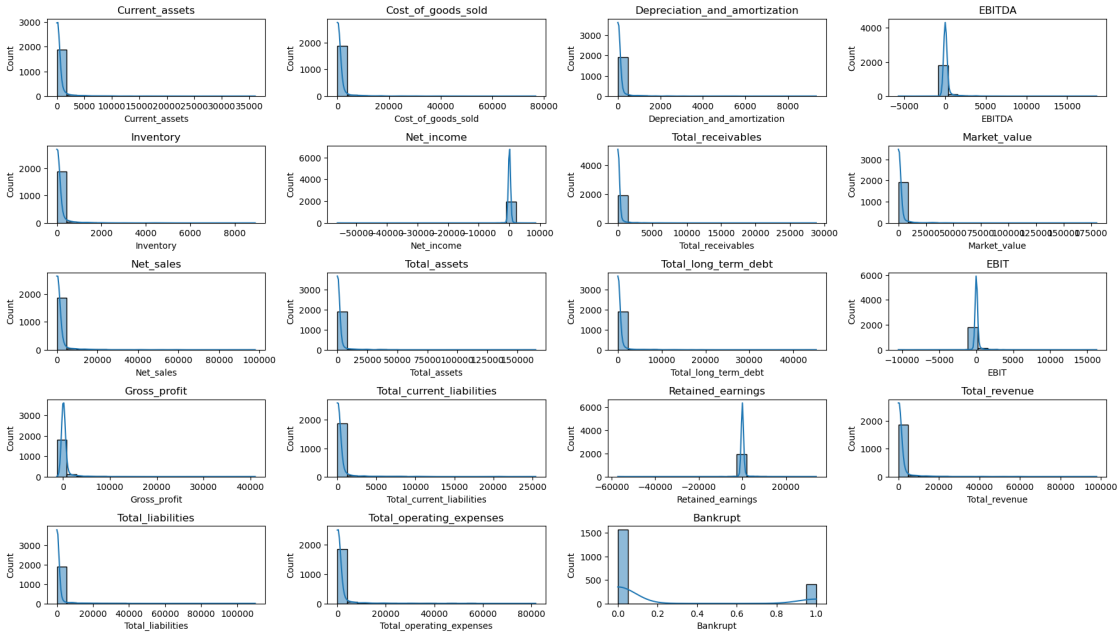
- 20.88% of the companies in the dataset are defaulters (i.e., bankrupt)
- The remaining 79.12% are non-defaulters.

[19]:
```
#Get boxplots for all the numerical columns
numeric_columns = df.select_dtypes(include=np.number).columns.tolist()

plt.figure(figsize=(18, 30))

for i, variable in enumerate(numeric_columns):
    plt.subplot(15, 4, i + 1)
    sns.boxplot(data=df, x=variable)  ## Complete the code to get boxplots for␣
 ↪all numerical columns
    plt.tight_layout()
    plt.title(variable)
```



- Most of the numerical columns have extreme outliers visible as points far from the whiskers of the boxplots.
- Net_income and Retained_earnings show heavy negative skewness.
- The Bankrupt column does not have outliers, as it is a binary variable (0 or 1).

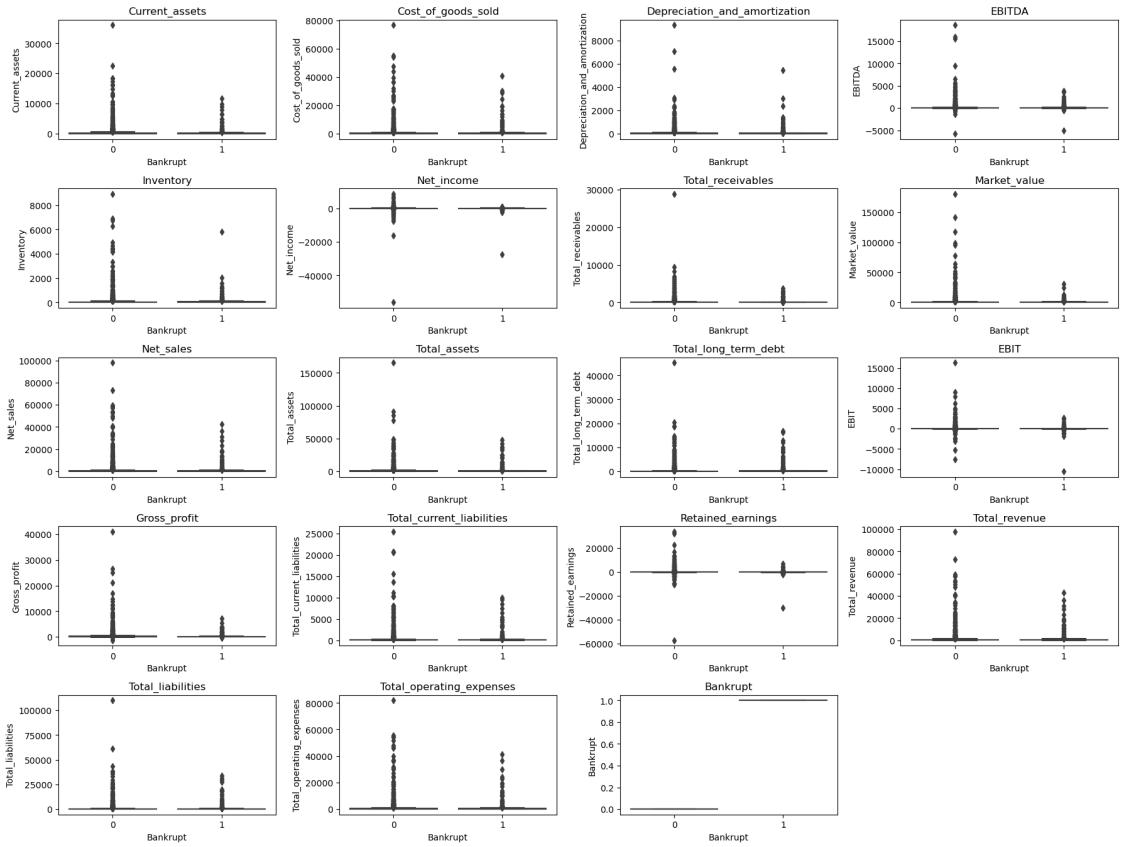
[90]:
```
#Get distplot for all the numerical columns
numeric_columns = df.select_dtypes(include=np.number).columns.tolist()

plt.figure(figsize=(18, 30))

for i, variable in enumerate(numeric_columns):
    plt.subplot(15, 4, i + 1)
    sns.histplot(data=df, x=variable, kde=True, bins=20, edgecolor='black')  ##␣
 ↪Complete the code to get histplot for all numerical columns in the data
    plt.tight_layout()
    plt.title(variable)
```



- Most of the columns exhibit high positive skewness.
- The long tails in many distributions indicate the presence of outliers, which were also shown in the boxplots.
- The Bankrupt histogram shows that most companies are non-bankrupt (label 0)

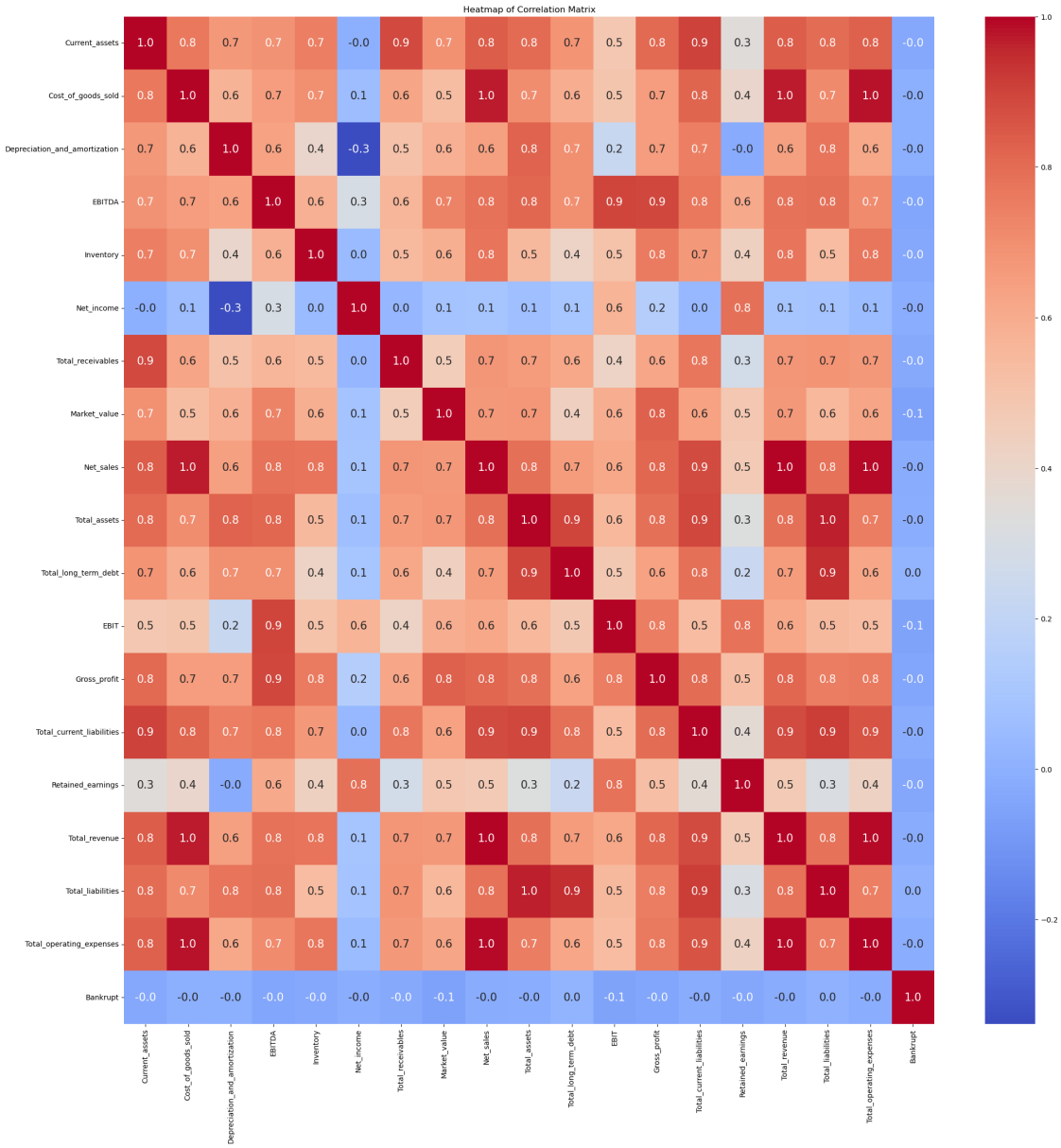
6.2 Bivariate Analysis

[22]:
```
#Get boxplots for all the numerical columns
numeric_columns = df.select_dtypes(include=np.number).columns.tolist()

plt.figure(figsize=(18, 40))

for i, variable in enumerate(numeric_columns):
    plt.subplot(15, 4, i + 1)
    sns.boxplot(x="Bankrupt", y=variable, data=df)  ## Complete the code to get␣
 ↪boxplot of all variables with Default column in the data
    plt.tight_layout()
    plt.title(variable)
plt.show()
```



- Some features, show visible differences in their distribution between bankrupt and non-bankrupt companies- Net_income, EBITDA, Retained_earnings, and Gross_profit.
- All features exhibit numerous outliers for both classes (bankrupt and non-bankrupt).
- Features like Net_income and Retained_earnings have a wide range of negative values for bankrupt companies, highlighting financial distress

[23]:
```
# Calculate the correlation matrix
corr_matrix = df.corr(numeric_only=True)

# Create a heatmap of the correlation matrix
plt.figure(figsize=(25, 25))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".1f",␣
 ↪annot_kws={"size": 15})
plt.title('Heatmap of Correlation Matrix')
plt.show()
```



6.3 Key Observations:

6.3.1 Strong Positive Correlations:

- Features like Current_assets, Total_assets, Net_sales, Market_value, and Total_revenue are strongly positively correlated with each other (correlation values close to 1). This indicates that these variables are highly interdependent and may provide redundant information in the model.

6.3.2 Weak or No Correlation with Bankrupt:

- Most features show little to no correlation with the target variable Bankrupt (correlation values close to 0). This suggests that:
- A linear relationship between these features and Bankrupt is weak or non-existent.
- Non-linear models might perform better in capturing the relationships.

6.3.3 Negative Correlations:

- Features like Net_income, Retained_earnings, and EBIT show negative correlations with Bankrupt. This indicates that higher values of these features are associated with a lower likelihood of bankruptcy.

7 Data Preprocessing

7.1 Outliers Check

[24]:
```
outliers_count = {}

# Iterate over each column in the DataFrame
for column in df.columns:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR  ## Fill the blank with correct value for␣
 ↪getting lower_bound
    upper_bound = Q3 + 1.5 * IQR  ## Fill the blank with correct value for␣
 ↪getting upper_bound

    outliers = df[(df[column] < lower_bound) | (df[column] > upper_bound)]
    outliers_count[column] = len(outliers)

print("Number of outliers in each column:")
pd.DataFrame([{'Column': column, 'No. of outliers': outliers} for column,␣
 ↪outliers in outliers_count.items()])
```

```
Number of outliers in each column:
```

[24]:

	Column	No. of outliers
0	Current_assets	279
1	Cost_of_goods_sold	311
2	Depreciation_and_amortization	302
3	EBITDA	335
4	Inventory	298
5	Net_income	485
6	Total_receivables	307
7	Market_value	278

```
8                        Net_sales              296
9                     Total_assets              310
10           Total_long_term_debt              332
11                           EBIT              413
12                   Gross_profit              280
13      Total_current_liabilities              314
14              Retained_earnings              408
15                  Total_revenue              296
16              Total_liabilities              312
17       Total_operating_expenses              296
18                       Bankrupt              414
```

7.2 Data Preparation for Modeling

[27]:
```
# Seperating target variable from the rest of the data
df_X = df.drop(['Bankrupt'], axis = 1)
df_y = df['Bankrupt']
```

[28]:
```
#Splitting the data for training and testing
X_train, X_test, y_train, y_test = train_test_split(df_X, df_y, test_size=0.25,␣
↪random_state=42, stratify = df_y)  ## Complete the code to split the data␣
↪into train and test in the ratio 75:25
```

7.3 Missing Values Detection

[29]:
```
# Check missing values
X_train.isnull().sum()  ## Complete the code to get the number of null or NaN␣
↪values in each column
```

[29]:
```
Current_assets                  0
Cost_of_goods_sold              0
Depreciation_and_amortization   0
EBITDA                          0
Inventory                       0
Net_income                      0
Total_receivables               0
Market_value                    0
Net_sales                       0
Total_assets                    0
Total_long_term_debt            0
EBIT                            0
Gross_profit                    0
Total_current_liabilities       0
Retained_earnings               0
Total_revenue                   0
Total_liabilities               0
Total_operating_expenses        0
```

```
dtype: int64
```

[30]:
```
# Check missing values
X_test.isnull().sum()
```

[30]:
```
Current_assets                  0
Cost_of_goods_sold              0
Depreciation_and_amortization   0
EBITDA                          0
Inventory                       0
Net_income                      0
Total_receivables               0
Market_value                    0
Net_sales                       0
Total_assets                    0
Total_long_term_debt            0
EBIT                            0
Gross_profit                    0
Total_current_liabilities       0
Retained_earnings               0
Total_revenue                   0
Total_liabilities               0
Total_operating_expenses        0
dtype: int64
```

[32]:
```
print("No. of missing values in training data:", X_train.isnull().sum().sum())
print("No. of missing values in test data:", X_test.isnull().sum().sum())
```

```
No. of missing values in training data: 0
No. of missing values in test data: 0
```

7.4 Scaling the Data

[33]:
```
#Scaling of features is done to bring all the features to the same scale.
sc = StandardScaler()

X_train_scaled = pd.DataFrame(sc.fit_transform(X_train), columns=X_train.
 ↪columns)
X_test_scaled = pd.DataFrame(sc.transform(X_test), columns=X_test.columns)  ##␣
 ↪Complete the code to scale X_test to the same scale as X_train
```

[34]:
```
X_train_scaled.head()
```

[34]:
```
   Current_assets  Cost_of_goods_sold  Depreciation_and_amortization  \
0       -0.159768           -0.142125                      -0.234780
1       -0.274514           -0.236003                      -0.261220
2       -0.175941           -0.160988                      -0.136617
3       -0.274668           -0.236195                      -0.261416
```

```
4       -0.272938           -0.235566                       -0.252166

     EBITDA  Inventory  Net_income  Total_receivables  Market_value  \
0 -0.063513  -0.113882    0.099548          -0.072098     -0.091104
1 -0.199194  -0.237175    0.000488          -0.191910     -0.198944
2 -0.075109  -0.078514    0.054720          -0.122962     -0.138230
3 -0.198389  -0.237910    0.001816          -0.192215     -0.199350
4 -0.220011  -0.234932   -0.043588          -0.192093     -0.190211

   Net_sales  Total_assets  Total_long_term_debt      EBIT  Gross_profit  \
0  -0.132506     -0.167604             -0.217799  0.022548     -0.074157
1  -0.253588     -0.267951             -0.263127 -0.139896     -0.232395
2  -0.155378     -0.172370             -0.173698 -0.035852     -0.101208
3  -0.253852     -0.267943             -0.263127 -0.138775     -0.232771
4  -0.253589     -0.264987             -0.257469 -0.170631     -0.233494

   Total_current_liabilities  Retained_earnings  Total_revenue  \
0                  -0.203199           0.040036      -0.132506
1                  -0.251483          -0.120986      -0.253588
2                  -0.162482           0.020665      -0.155378
3                  -0.252070          -0.121302      -0.253852
4                  -0.250024          -0.189633      -0.253589

   Total_liabilities  Total_operating_expenses
0          -0.224333                 -0.139028
1          -0.262956                 -0.253171
2          -0.186075                 -0.162921
3          -0.263176                 -0.253603
4          -0.260105                 -0.249713
```

[35]:
```
X_test_scaled.head()
```

[35]:
```
   Current_assets  Cost_of_goods_sold  Depreciation_and_amortization  \
0        0.961504            0.178218                       1.739125
1       -0.263573           -0.229975                      -0.189535
2       -0.272713           -0.234976                      -0.258923
3       -0.265103           -0.230545                      -0.251250
4        0.759964            0.384240                       0.441011

     EBITDA  Inventory  Net_income  Total_receivables  Market_value  \
0  0.534310   0.435299   -0.094970           0.491741      0.398906
1 -0.198939  -0.237910   -0.093674          -0.177780     -0.198400
2 -0.199232  -0.237679   -0.003717          -0.190784     -0.198345
3 -0.177672  -0.225244    0.016272          -0.182056     -0.185388
4  0.824896   1.277753    0.613444           0.699953      0.449776

   Net_sales  Total_assets  Total_long_term_debt      EBIT  Gross_profit  \
```

```
0   0.381419      0.604897              0.140906 -0.085077      0.793798
1  -0.242869     -0.262008             -0.262274 -0.171345     -0.212647
2  -0.252523     -0.266531             -0.261575 -0.140962     -0.231508
3  -0.243520     -0.255296             -0.253015 -0.116689     -0.213333
4   0.614304      0.551349              0.307429  0.863365      1.034145

   Total_current_liabilities  Retained_earnings  Total_revenue  \
0                   0.595817           0.761951       0.381419
1                  -0.234362          -0.158177      -0.242869
2                  -0.248233          -0.133136      -0.252523
3                  -0.248004          -0.095540      -0.243520
4                   0.559972           2.016841       0.614304

   Total_liabilities  Total_operating_expenses
0           0.243339                  0.341795
1          -0.255829                 -0.241113
2          -0.261111                 -0.251962
3          -0.256529                 -0.245381
4           0.416513                  0.556410
```

8 Model Building

8.1 Model Evaluation Criterion

8.2 *Metric of Choice*

[47]:
```
# defining a function to compute different metrics to check performance of a␣
↪classification model built using sklearn


def model_performance_classification(model, predictors, target, threshold = 0.
↪5):
    """
    Function to compute different metrics to check classification model␣
↪performance

    model: classifier
    predictors: independent variables
    target: dependent variable
    """

    # predicting using the independent variables
    y_pred = model.predict(predictors)

    if len(list(set(y_pred))) != 2:
        y_prob_pred = model.predict(predictors)
```

```
        y_pred=[]
        for i in range(0,len(y_prob_pred)):
            if np.array(y_prob_pred)[i] > threshold:
                a=1
            else:
                a=0
            y_pred.append(a)
    else:
        pass

    acc = accuracy_score(target, y_pred)  # to compute Accuracy
    recall = recall_score(target, y_pred)  # to compute Recall
    precision = precision_score(target, y_pred)  # to compute Precision
    f1 = f1_score(target, y_pred)  # to compute F1-score

    # creating a dataframe of metrics
    df_perf = pd.DataFrame(
        {"Accuracy": acc, "Recall": recall, "Precision": precision, "F1": f1,},
        index=[0],
    )

    return df_perf
```

[48]:
```
def model_confusion_matrix(model, predictors, target, threshold = 0.5):
    """
    To plot the confusion_matrix with percentages

    model: classifier
    predictors: independent variables
    target: dependent variable
    """
    y_pred = model.predict(predictors)
    if len(list(set(y_pred))) != 2:
        y_prob_pred = model.predict(predictors)

        y_pred=[]
        for i in range(0,len(y_prob_pred)):
            if np.array(y_prob_pred)[i] > threshold:
                a=1
            else:
                a=0
            y_pred.append(a)
    else:
        pass

    cm = confusion_matrix(target, y_pred)
    labels = np.asarray(
```

```
        [
            ["{0:0.0f}".format(item) + "\n{0:.2%}".format(item / cm.flatten().
↪sum())]
            for item in cm.flatten()
        ]
    ).reshape(2, 2)

    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=labels, fmt="")
    plt.ylabel("True label")
    plt.xlabel("Predicted label")
```

8.3 Logistic Regression

[49]:
```
# Adding constant to data for Logistic Regression
X_train_with_intercept = SM.add_constant(X_train_scaled)
X_test_with_intercept = SM.add_constant(X_test_scaled)
```

[50]:
```
X_train_with_intercept.head()
```

[50]:
```
   const  Current_assets  Cost_of_goods_sold  Depreciation_and_amortization  \
0    1.0       -0.159768           -0.142125                      -0.234780
1    1.0       -0.274514           -0.236003                      -0.261220
2    1.0       -0.175941           -0.160988                      -0.136617
3    1.0       -0.274668           -0.236195                      -0.261416
4    1.0       -0.272938           -0.235566                      -0.252166

     EBITDA  Inventory  Net_income  Total_receivables  Market_value  \
0 -0.063513  -0.113882    0.099548          -0.072098     -0.091104
1 -0.199194  -0.237175    0.000488          -0.191910     -0.198944
2 -0.075109  -0.078514    0.054720          -0.122962     -0.138230
3 -0.198389  -0.237910    0.001816          -0.192215     -0.199350
4 -0.220011  -0.234932   -0.043588          -0.192093     -0.190211

   Net_sales  Total_assets  Total_long_term_debt      EBIT  Gross_profit  \
0  -0.132506     -0.167604             -0.217799  0.022548     -0.074157
1  -0.253588     -0.267951             -0.263127 -0.139896     -0.232395
2  -0.155378     -0.172370             -0.173698 -0.035852     -0.101208
3  -0.253852     -0.267943             -0.263127 -0.138775     -0.232771
4  -0.253589     -0.264987             -0.257469 -0.170631     -0.233494

   Total_current_liabilities  Retained_earnings  Total_revenue  \
0                  -0.203199           0.040036      -0.132506
1                  -0.251483          -0.120986      -0.253588
2                  -0.162482           0.020665      -0.155378
3                  -0.252070          -0.121302      -0.253852
4                  -0.250024          -0.189633      -0.253589
```

```
   Total_liabilities  Total_operating_expenses
0          -0.224333                 -0.139028
1          -0.262956                 -0.253171
2          -0.186075                 -0.162921
3          -0.263176                 -0.253603
4          -0.260105                 -0.249713
```

[51]:
```python
y_train.reset_index(inplace = True, drop = True)
```

[52]:
```python
LogisticReg = SM.Logit(y_train, X_train_with_intercept).fit(method = 'bfgs')
print(LogisticReg.summary())
```

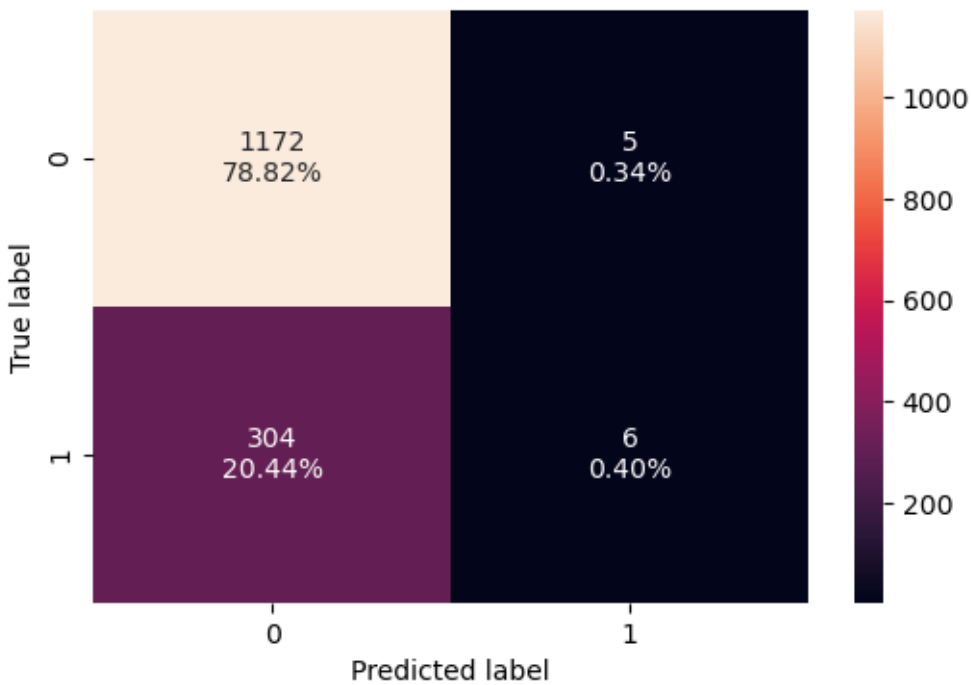
```
         Current function value: 0.497065
         Iterations: 35
         Function evaluations: 37
         Gradient evaluations: 37
                           Logit Regression Results
==============================================================================
Dep. Variable:               Bankrupt   No. Observations:                 1487
Model:                          Logit   Df Residuals:                     1472
Method:                           MLE   Df Model:                           14
Date:                Sun, 01 Dec 2024   Pseudo R-squ.:                 0.02903
Time:                        16:46:57   Log-Likelihood:                -739.14
converged:                       False   LL-Null:                       -761.24
Covariance Type:            nonrobust   LLR p-value:                 5.497e-05
==============================================================================
=================
                                    coef    std err          z      P>|z|
[0.025      0.975]
------------------------------------------------------------------------------
-----------------
const                             -1.4992        nan        nan        nan
nan         nan
Current_assets                    -0.1732        nan        nan        nan
nan         nan
Cost_of_goods_sold                -0.1164        nan        nan        nan
nan         nan
Depreciation_and_amortization      0.0138        nan        nan        nan
nan         nan
EBITDA                            -0.2657        nan        nan        nan
nan         nan
Inventory                         -0.1482        nan        nan        nan
nan         nan
Net_income                         0.3196        nan        nan        nan
nan         nan
Total_receivables                 -0.1693        nan        nan        nan
nan         nan
```

```
Market_value                     -2.0765        nan        nan        nan
nan         nan
Net_sales                         0.0723        nan        nan        nan
nan         nan
Total_assets                      0.9281        nan        nan        nan
nan         nan
Total_long_term_debt             -0.1962        nan        nan        nan
nan         nan
EBIT                             -0.3472        nan        nan        nan
nan         nan
Gross_profit                      0.5279        nan        nan        nan
nan         nan
Total_current_liabilities        -0.4953        nan        nan        nan
nan         nan
Retained_earnings                 0.0105        nan        nan        nan
nan         nan
Total_revenue                     0.0723        nan        nan        nan
nan         nan
Total_liabilities                 0.5868        nan        nan        nan
nan         nan
Total_operating_expenses          0.1257        nan        nan        nan
nan         nan
==============================================================================
=================
```

- Dependent Variable: Bankrupt.
- Number of Observations: 1487 observations.
- Pseudo R-squared: 0.02903, indicating a very low explanatory power of the model.
- Log-Likelihood: -739.14, which is a measure of the model's fit. A higher (less negative) value indicates better fit, but this value appears suboptimal.
- LLR p-value: 5.497e-05, indicating the model as a whole is statistically significant.
- Positive coefficients (e.g., Total_liabilities, Total_operating_expenses) suggest that an increase in these features is associated with a higher probability of bankruptcy.
- Negative coefficients (e.g., Market_value, Net_income) suggest that an increase in these features is associated with a lower probability of bankruptcy.

8.3.1 Logistic Regression Model - Training Performance

[53]:
```
model_confusion_matrix(LogisticReg, X_train_with_intercept, y_train)
```



True Negatives (Top Left - 1172):

- 1172 cases where the model correctly predicted not bankrupt (0). Represents 78.82% of the total data, indicating that the model is very good at predicting the majority class (non-bankrupt). #### False Positives (Top Right - 5):
- 5 cases where the model incorrectly predicted bankrupt (1). Represents 0.34% of the data, showing very few incorrect predictions for the non-bankrupt class. #### False Negatives (Bottom Left - 304):
- 304 cases where the model incorrectly predicted not bankrupt (0). Represents 20.44% of the data, indicating a significant number of missed bankruptcies. #### True Positives (Bottom Right - 6):
- 6 cases where the model correctly predicted bankrupt (1). Represents only 0.40% of the data, showing the model's poor ability to detect bankrupt companies.

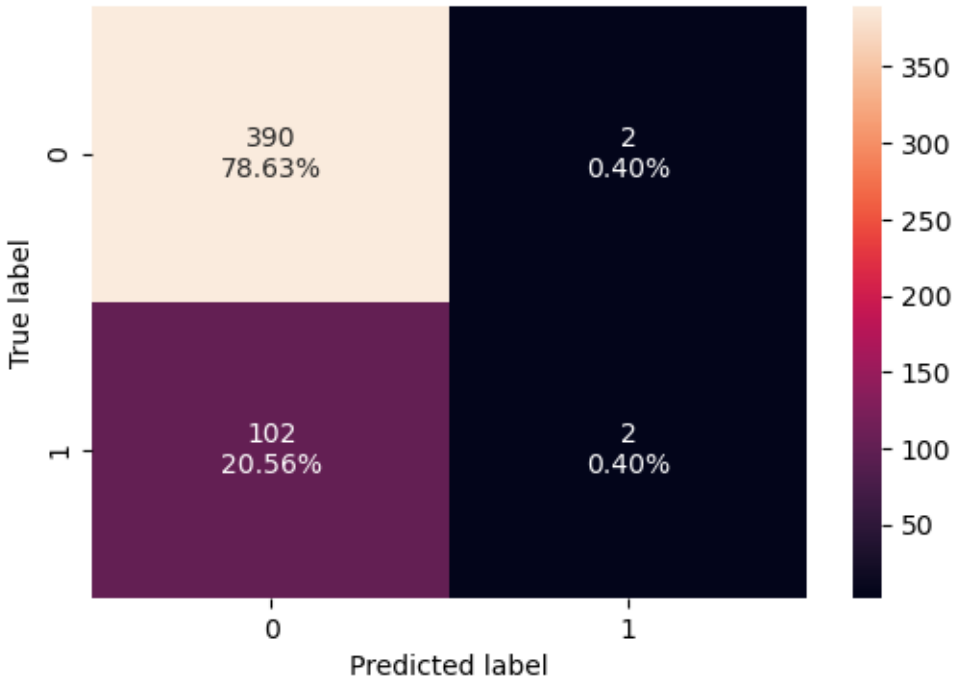
[54]:
```
logistic_regression_perf_train = model_performance_classification(LogisticReg,␣
↪X_train_with_intercept, y_train)
logistic_regression_perf_train
```

[54]:

	Accuracy	Recall	Precision	F1
0	0.792199	0.019355	0.545455	0.037383

8.3.2 Logistic Regression Model - Test Performance

[58]:
```
model_confusion_matrix(LogisticReg, X_test_with_intercept, y_test)  ## Complete␣
↪the code to create confusion matrix for test data
```



True Negatives (Top Left - 390):

- 390 cases where the model correctly predicted not bankrupt (0). Represents 78.63% of the total data, indicating that the model performs well on the majority class (non-bankrupt). #### False Positives (Top Right - 2):
- 2 cases where the model incorrectly predicted bankrupt (1). Represents 0.40% of the total data, showing that the model rarely misclassifies non-bankrupt companies as bankrupt. #### False Negatives (Bottom Left - 102):
- 102 cases where the model incorrectly predicted not bankrupt (0). Represents 20.56% of the total data, highlighting the model's difficulty in correctly identifying bankrupt companies. #### True Positives (Bottom Right - 2):
- 2 cases where the model correctly predicted bankrupt (1). Represents only 0.40% of the total data, showing the model's poor performance in identifying the minority class.

[59]:
```
logistic_regression_perf_test = model_performance_classification(LogisticReg,␣
↪X_test_with_intercept, y_test)  ## Complete the code to check performance on␣
↪test data
logistic_regression_perf_test
```

[59]:

	Accuracy	Recall	Precision	F1
0	0.790323	0.019231	0.5	0.037037

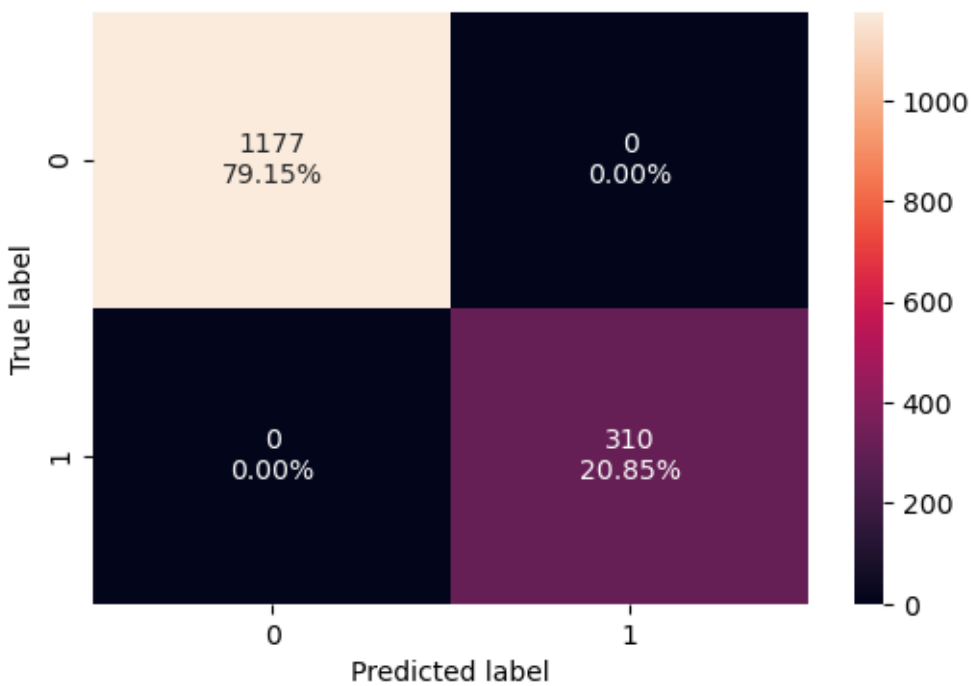
8.4 Random Forest

[97]:
```
rf_classifier = RandomForestClassifier(random_state = 42)
rf_model = rf_classifier.fit(X_train, y_train)
rf_model
```

[97]: RandomForestClassifier(random_state=42)

8.4.1 Random Forest Model - Training Performance

[98]:
```
model_confusion_matrix(rf_model, X_train, y_train)
```



True Negatives (Top Left - 1177):

- 1177 cases where the model correctly predicted not bankrupt (0). Represents 79.15% of the total data. #### False Positives (Top Right - 0):
- 0 cases where the model incorrectly predicted bankrupt (1). Indicates no false alarms for the non-bankrupt class. #### False Negatives (Bottom Left - 0):
- 0 cases where the model incorrectly predicted not bankrupt (0). #### True Positives (Bottom Right - 310):
- 310 cases where the model correctly predicted bankrupt (1). Represents 20.85% of the total data.

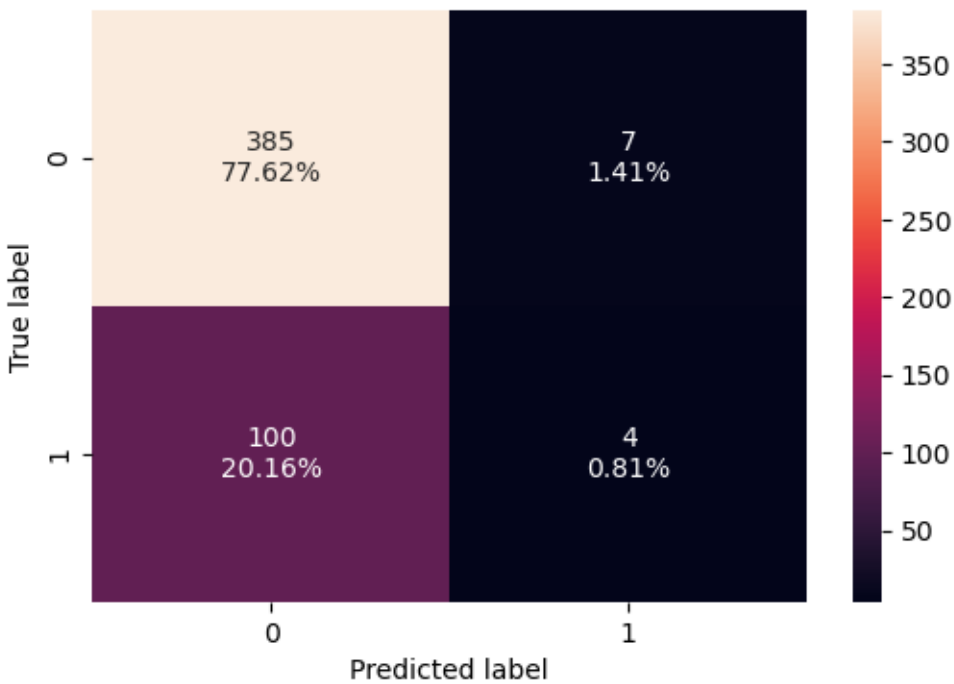
[99]:
```
random_forest_perf_train = model_performance_classification(rf_model, X_train,␣
↪y_train)
random_forest_perf_train
```

[99]:

	Accuracy	Recall	Precision	F1
0	1.0	1.0	1.0	1.0

8.4.2 Random Forest Model - Test Performance

[100]:
```
model_confusion_matrix(rf_model, X_test, y_test)  ## Complete the code to␣
↪create confusion matrix for test data
```



True Negatives (Top Left - 385):

- 385 cases where the model correctly predicted not bankrupt (0). Represents 77.62% of the total predictions, showing the model performs reasonably well on the majority class. #### False Positives (Top Right - 7):
- 7 cases where the model incorrectly predicted bankrupt (1). Represents 1.41% of the total predictions, indicating relatively few incorrect predictions for the non-bankrupt class. #### False Negatives (Bottom Left - 100):
- 100 cases where the model incorrectly predicted not bankrupt (0). Represents 20.16% of the total predictions, highlighting a significant number of missed bankruptcies. #### True Positives (Bottom Right - 4):
- 4 cases where the model correctly predicted bankrupt (1). Represents only 0.81% of the total predictions, indicating the model struggles to identify bankrupt companies.

[101]:
```
random_forest_perf_test = model_performance_classification(rf_model, X_test,␣
↪y_test)  ## Complete the code to check performance on test data
random_forest_perf_test
```

[101]:

	Accuracy	Recall	Precision	F1
0	0.784274	0.038462	0.363636	0.069565

9 Model Performance Improvement

9.1 Model Performance Improvement - Logistic Regression

[102]:
```
def calculate_vif(idf):
    """
    Calculate Variance Inflation Factor (VIF) for each variable in a DataFrame.

    Parameters:
    df (DataFrame): Input DataFrame containing numerical variables.

    Returns:
    vif_df (DataFrame): DataFrame containing variable names and their␣
↪corresponding VIF values.
    """
    variables = idf.values
    vif_df = pd.DataFrame()
    vif_df["Variable"] = idf.columns
    vif_df["VIF"] = [variance_inflation_factor(variables, i) for i in range(idf.
↪shape[1])]
    return vif_df
```

[103]:
```
# Call the function to calculate VIF
vif_result = calculate_vif(X_train_scaled)  ## Complete the code to calculate␣
↪VIF for the scaled X_train data

print("Variance Inflation Factors:")
vif_result
```

```
Variance Inflation Factors:
```

[103]:

	Variable	VIF
0	Inventory	2.900286
1	Net_income	2.179278
2	Total_receivables	1.960079
3	Market_value	2.476668
4	Total_long_term_debt	2.167227
5	Retained_earnings	4.656823
6	Total_operating_expenses	4.268428

[68]:
```
# Dropping columns with VIF > 5 iteratively
while vif_result['VIF'].max() > 5:
    high_vif_column = vif_result.loc[vif_result['VIF'].idxmax(), 'Variable']
    print(f"Dropping {high_vif_column} due to high VIF")
    X_train_scaled.drop(columns=high_vif_column, inplace=True)
    X_test_scaled.drop(columns=high_vif_column, inplace=True)
    vif_result = calculate_vif(X_train_scaled)
```

```
Dropping Cost_of_goods_sold due to high VIF
Dropping Depreciation_and_amortization due to high VIF
Dropping EBITDA due to high VIF
Dropping Net_sales due to high VIF
Dropping Total_revenue due to high VIF
Dropping Total_liabilities due to high VIF
Dropping Current_assets due to high VIF
Dropping Gross_profit due to high VIF
Dropping Total_assets due to high VIF
Dropping Total_current_liabilities due to high VIF
Dropping EBIT due to high VIF
```

[69]:
```
print(f"There are {X_train_scaled.shape[0]} records in the Train Dataset and␣
↪{X_test_scaled.shape[0]} records in the Test Dataset. \nBoth the datsets␣
↪have {X_train_scaled.shape[1]} variables.")
```

```
There are 1487 records in the Train Dataset and 496 records in the Test Dataset.
Both the datsets have 7 variables.
```

[70]:
```
X_train_new_with_intercept = SM.add_constant(X_train_scaled)
X_test_new_with_intercept = SM.add_constant(X_test_scaled)
```

[71]:
```
# Retraining Logistic Regression Model with new data
LogisticReg_improved = SM.Logit(y_train, X_train_new_with_intercept).fit(method␣
↪= 'bfgs')
print(LogisticReg_improved.summary())
```

```
         Current function value: 0.501131
         Iterations: 35
         Function evaluations: 36
         Gradient evaluations: 36
                           Logit Regression Results
==============================================================================
Dep. Variable:               Bankrupt   No. Observations:                 1487
Model:                          Logit   Df Residuals:                     1479
Method:                           MLE   Df Model:                            7
Date:                Sun, 01 Dec 2024   Pseudo R-squ.:                 0.02109
Time:                        17:22:32   Log-Likelihood:                -745.18
converged:                       False   LL-Null:                       -761.24
Covariance Type:            nonrobust   LLR p-value:                 3.879e-05
==============================================================================
```

```
============
                              coef    std err          z      P>|z|      [0.025
0.975]
------------------------------------------------------------------------------
------------
const                       -1.4801      0.084    -17.714      0.000      -1.644
-1.316
Inventory                   -0.0740      0.167     -0.442      0.658      -0.402
0.254
Net_income                  -0.0296      0.186     -0.159      0.874      -0.394
0.335
Total_receivables           -0.3465      0.292     -1.187      0.235      -0.919
0.226
Market_value                -1.6803      0.526     -3.192      0.001      -2.712
-0.649
Total_long_term_debt         0.4421      0.125      3.529      0.000       0.197
0.688
Retained_earnings            0.2073      0.238      0.872      0.383      -0.259
0.673
Total_operating_expenses     0.2341      0.191      1.226      0.220      -0.140
0.608
==============================================================================
============
```

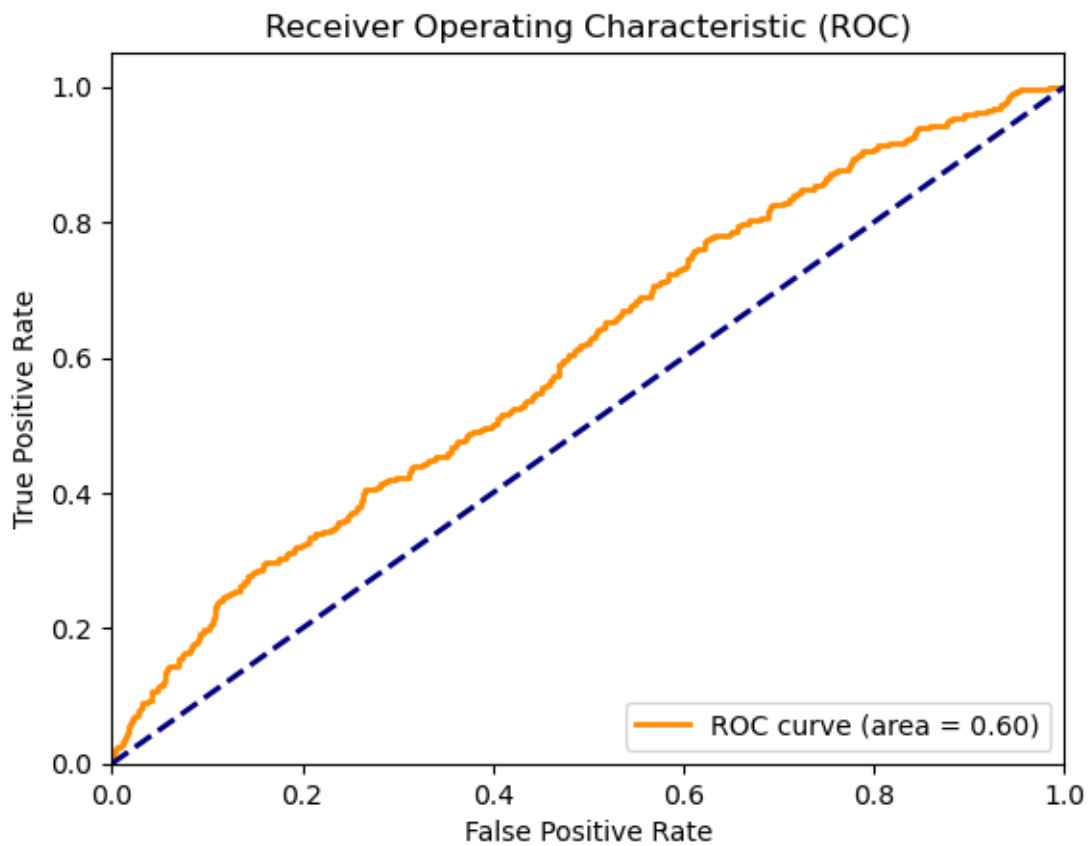
- Dependent Variable: Bankrupt, indicating the binary target variable.
- Number of Observations: 1487, meaning 1487 data points were used to fit the model.
- Pseudo R-squared: 0.02109, which is very low. This suggests that the model explains only about 2.1% of the variance in the target variable. The model might not capture the relationship well.
- Log-Likelihood: -745.18, indicating the fit of the model. A higher value would imply a better fit.
- LLR p-value: 3.879e-05, which is statistically significant. This indicates the overall model is meaningful. #### Positive Coefficients:
- Total_long_term_debt (0.4421): Companies with higher long-term debt are more likely to go bankrupt, as indicated by a positive coefficient.
- Retained_earnings (0.2073): Higher retained earnings slightly increase the probability of bankruptcy.
- Total_operating_expenses (0.2341): Higher operating expenses are positively associated with bankruptcy. #### Negative Coefficients:
- Inventory (-0.0740): Higher inventory reduces the likelihood of bankruptcy.
- Net_income (-0.0296): Higher net income slightly decreases the likelihood of bankruptcy.
- Total_receivables (-0.3465): Companies with higher receivables are less likely to go bankrupt.
- Market_value (-1.6803): Higher market value strongly decreases the probability of bankruptcy. #### Significant Predictors ($p < 0.05$):
- Market_value (p=0.001): Companies with higher market value are significantly less likely to go bankrupt.
- Total_long_term_debt (p=0.000): Higher long-term debt significantly increases the likelihood of bankruptcy. #### Non-Significant Predictors ($p > 0.05$):

- Variables like Inventory, Net_income, Total_receivables, Retained_earnings, and Total_operating_expenses have high p-values, suggesting they do not contribute significantly to the model.

[72]:
```
# Finding Optimal Threshold value
logit_y_pred = LogisticReg_improved.predict(X_train_new_with_intercept)
fpr, tpr, thresholds = roc_curve(y_train, logit_y_pred)
optimal_idx = np.argmax(tpr - fpr)
optimal_threshold_logit = round(thresholds[optimal_idx], 3)
optimal_threshold_logit
```

[72]: 0.215

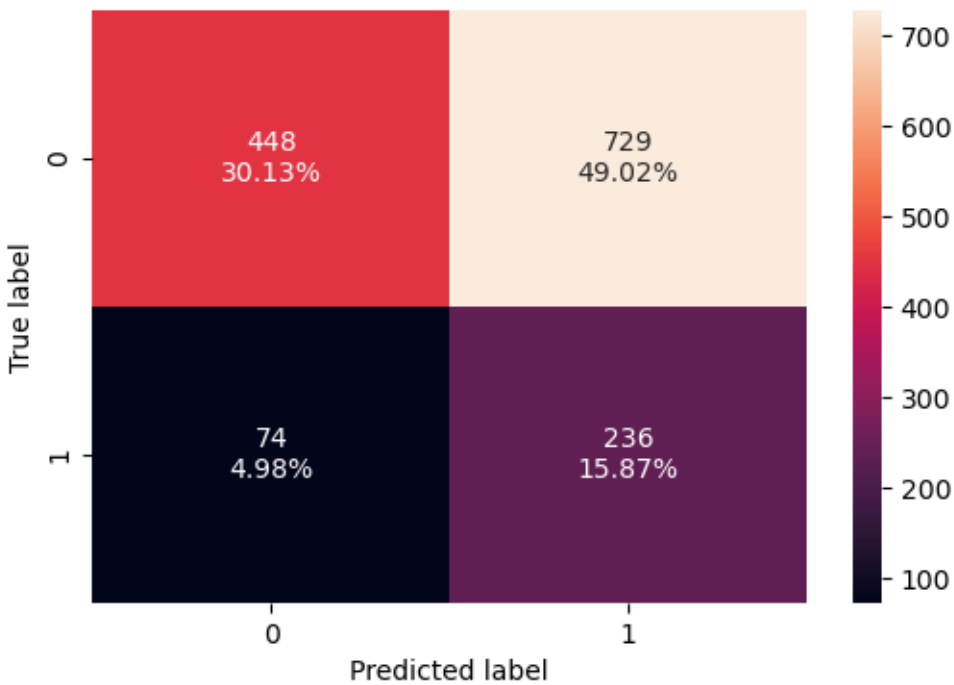
[73]:
```
roc_auc = roc_auc_score(y_train, logit_y_pred)  ## Complete the code to get␣
↪roc_auc score
# Plot ROC curve
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' %␣
↪roc_auc)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC)')
plt.legend(loc="lower right")
plt.show()
```



- The curve lies close to the diagonal line (the dashed line), which represents a random classifier.
- The AUC-ROC score is 0.60, which indicates poor performance. An AUC score closer to 1 represents excellent performance, while 0.5 indicates random guessing.

9.1.1 Logistic Regression Performance - Training Set

[74]:
```
model_confusion_matrix(LogisticReg_improved, X_train_new_with_intercept,␣
↪y_train, optimal_threshold_logit)
```



True Negatives (Top Left - 448):

- The model correctly predicted not bankrupt (0). Represents 30.13% of the total data. #### False Positives (Top Right - 729):
- The model incorrectly predicted bankrupt (1). Represents 49.02% of the total data, indicating a high rate of false alarms for the non-bankrupt class. #### False Negatives (Bottom Left - 74):
- The model incorrectly predicted not bankrupt (0). Represents 4.98% of the total data, showing a moderate number of missed bankruptcies. #### True Positives (Bottom Right - 236):
- The model correctly predicted bankrupt (1). Represents 15.87% of the total data.

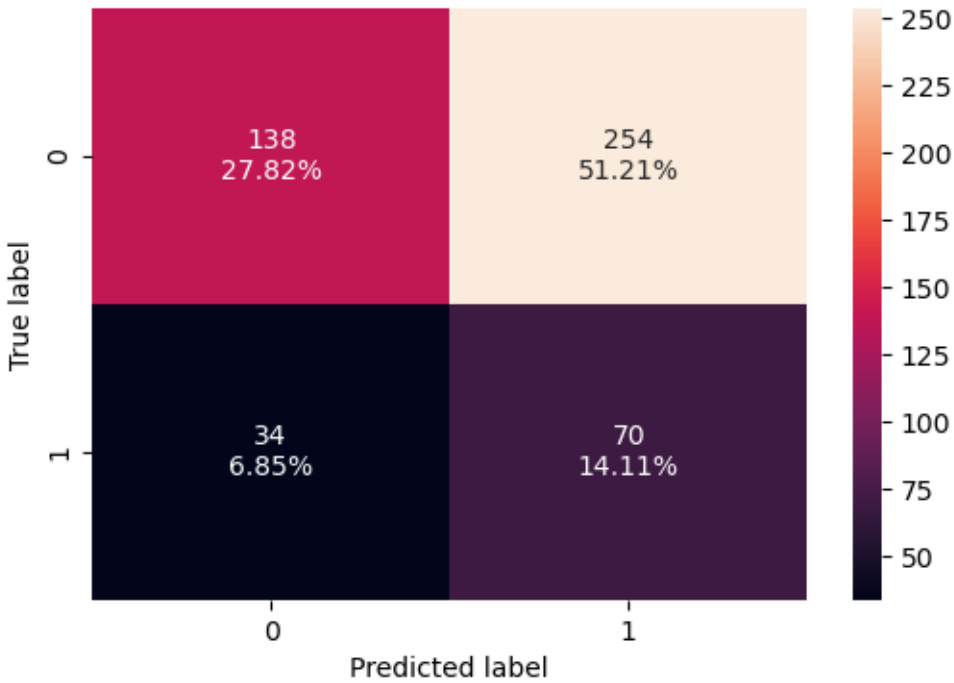
[75]:
```
logistic_regression_tuned_perf_train = model_performance_classification(
    LogisticReg_improved, X_train_new_with_intercept, y_train,␣
 ↪optimal_threshold_logit
)
logistic_regression_tuned_perf_train
```

[75]:

	Accuracy	Recall	Precision	F1
0	0.459987	0.76129	0.24456	0.370196

9.1.2 Logistic Regression Performance - Test Set

[76]:
```
model_confusion_matrix(LogisticReg_improved, X_test_new_with_intercept, y_test,␣
↪optimal_threshold_logit)  ## Complete the code to create confusion matrix␣
↪for test data
```



True Negatives (Top Left - 138):

- The model correctly predicted not bankrupt (0). Represents 27.82% of the total data. #### False Positives (Top Right - 254):
- The model incorrectly predicted bankrupt (1). Represents 51.21% of the total data, indicating a high false positive rate. #### False Negatives (Bottom Left - 34):
- The model incorrectly predicted not bankrupt (0). Represents 6.85% of the total data, showing a moderate number of missed bankruptcies. #### True Positives (Bottom Right - 70):
- The model correctly predicted bankrupt (1). Represents 14.11% of the total data.

[78]:
```
logistic_regression_tuned_perf_test =␣
↪model_performance_classification(LogisticReg_improved,␣
↪X_test_new_with_intercept, y_test, optimal_threshold_logit)  ## Complete the␣
↪code to check performance on test data
logistic_regression_tuned_perf_test
```

[78]:

	Accuracy	Recall	Precision	F1
0	0.419355	0.673077	0.216049	0.327103

9.2 Model Performance Improvement - Random Forest

Note: The parameter grid provided below is a sample grid. It can be updated if required.

[79]:
```
param_grid = {
    'n_estimators': [100, 150, 200],  # Number of trees in the forest
    'max_depth': [5, 6, 7, 8],     # Maximum depth of the trees
    'min_samples_split': [2, 4, 5, 7, 10],    # Minimum number of samples␣
 ↪required to split a node
    'min_samples_leaf': [5, 6, 7, 9],  # Minimum number of samples required at␣
 ↪each leaf node
}

rf_classifier = RandomForestClassifier(class_weight='balanced', random_state=42)

grid_search = GridSearchCV(
    estimator=rf_classifier,
    param_grid=param_grid,
    cv=5,
    scoring='recall',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("Best parameters:", grid_search.best_params_)
```

```
Best parameters: {'max_depth': 5, 'min_samples_leaf': 9, 'min_samples_split': 2,
'n_estimators': 100}
```

[80]:
```
# Access the best estimator directly if needed
best_rf_classifier = grid_search.best_estimator_
```

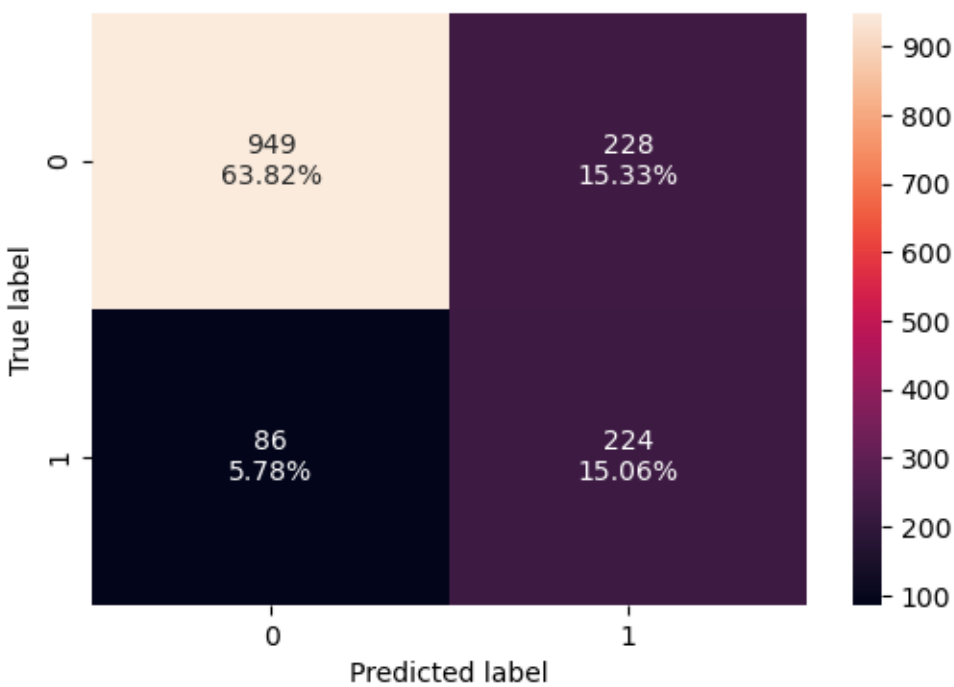
[81]:
```
params_used = best_rf_classifier.get_params()

# Print the parameters
print("Parameters used in the Random Forest Classifier:")
for param_name, param_value in params_used.items():
    print(f"{param_name}: {param_value}")
```

```
Parameters used in the Random Forest Classifier:
bootstrap: True
ccp_alpha: 0.0
class_weight: balanced
criterion: gini
max_depth: 5
max_features: sqrt
max_leaf_nodes: None
max_samples: None
```

```
min_impurity_decrease: 0.0
min_samples_leaf: 9
min_samples_split: 2
min_weight_fraction_leaf: 0.0
monotonic_cst: None
n_estimators: 100
n_jobs: None
oob_score: False
random_state: 42
verbose: 0
warm_start: False
```

9.2.1 Random Forest Performance - Training Set

[82]:
```
model_confusion_matrix(best_rf_classifier, X_train, y_train)  ## Complete the
↪code to create confusion matrix for training data
```



True Negatives (Top Left - 949):

- The model correctly predicted not bankrupt (0). Represents 63.82% of the total data, indicating a good performance in identifying the majority class. #### False Positives (Top Right - 228):
- The model incorrectly predicted bankrupt (1). Represents 15.33% of the total data, suggesting some overprediction for the minority class (Bankrupt). #### False Negatives (Bottom Left - 86):

- The model incorrectly predicted not bankrupt (0). Represents 5.78% of the total data, indicating a moderate number of missed bankruptcies. #### True Positives (Bottom Right - 224):
- The model correctly predicted bankrupt (1). Represents 15.06% of the total data, showing decent performance in identifying bankrupt companies.

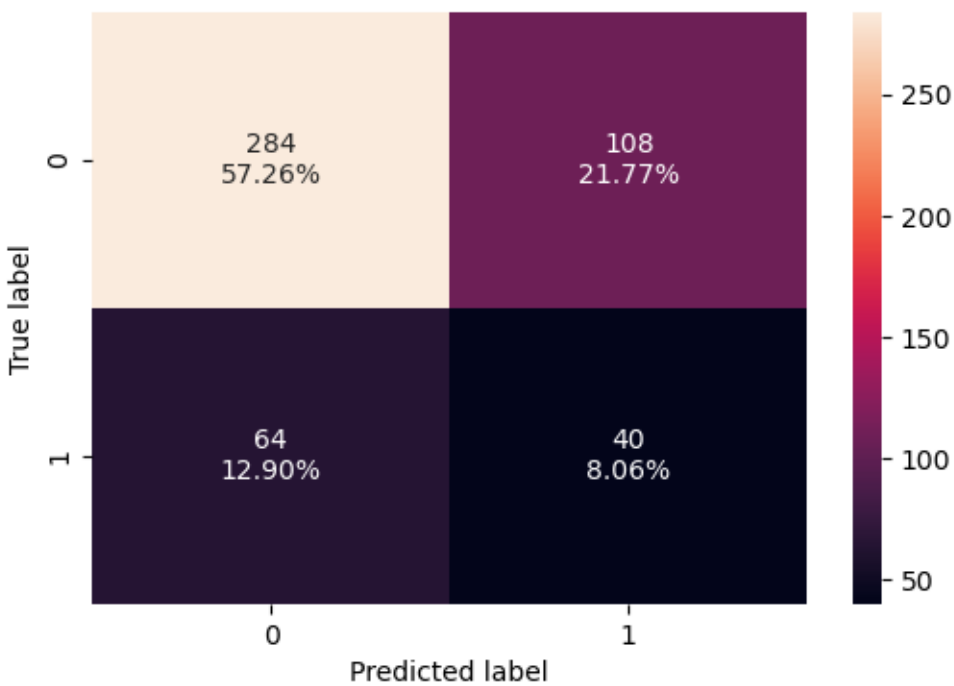
[83]:
```
random_forest_tuned_perf_train =␣
↪model_performance_classification(best_rf_classifier, X_train, y_train)  ##␣
↪Complete the code to check performance on training data
random_forest_tuned_perf_train
```

[83]:

	Accuracy	Recall	Precision	F1
0	0.788837	0.722581	0.495575	0.587927

9.2.2 Random Forest Performance - Test Set

[84]:
```
model_confusion_matrix(best_rf_classifier, X_test, y_test)  ## Complete the␣
↪code to create confusion matrix for test data
```



True Negatives (Top Left - 284):

- The model correctly predicted not bankrupt (0). Represents 57.26% of the total data, showing good performance in identifying the majority class. #### False Positives (Top Right - 108):
- The model incorrectly predicted bankrupt (1). Represents 21.77% of the total data, indicating a moderate false positive rate. #### False Negatives (Bottom Left - 64):

- The model incorrectly predicted not bankrupt (0). Represents 12.90% of the total data, highlighting a significant number of missed bankruptcies. #### True Positives (Bottom Right - 40):
- The model correctly predicted bankrupt (1). Represents 8.06% of the total data, showing limited success in identifying bankrupt companies.

[85]:
```
random_forest_tuned_perf_test =␣
↪model_performance_classification(best_rf_classifier, X_test, y_test)  ##␣
↪Complete the code to check performance on test data
random_forest_tuned_perf_test
```

[85]:

	Accuracy	Recall	Precision	F1
0	0.653226	0.384615	0.27027	0.31746

10 Model Comparison and Final Model Selection

[86]:
```
# training performance comparison

models_train_comp_df = pd.concat(
    [
        logistic_regression_perf_train.T,
        logistic_regression_tuned_perf_train.T,
        random_forest_perf_train.T,
        random_forest_tuned_perf_train.T,
    ],
    axis=1,
)
models_train_comp_df.columns = [
    "Logistic Regression",
    "Tuned Logistic Regression",
    "Random Forest",
    "Tuned Random Forest",
]
print("Training performance comparison:")
models_train_comp_df
```

```
Training performance comparison:
```

[86]:

	Logistic Regression	Tuned Logistic Regression	Random Forest	\
Accuracy	0.792199	0.459987	1.0	
Recall	0.019355	0.761290	1.0	
Precision	0.545455	0.244560	1.0	
F1	0.037383	0.370196	1.0	

	Tuned Random Forest
Accuracy	0.788837
Recall	0.722581

```
Precision              0.495575
F1                     0.587927
```

[87]:
```python
# testing performance comparison

models_test_comp_df = pd.concat(
    [
        logistic_regression_perf_test.T,
        logistic_regression_tuned_perf_test.T,
        random_forest_perf_test.T,
        random_forest_tuned_perf_test.T,
    ],
    axis=1,
)
models_test_comp_df.columns = [
    "Logistic Regression",
    "Tuned Logistic Regression",
    "Random Forest",
    "Tuned Random Forest",
]
print("Testing performance comparison:")
models_test_comp_df
```

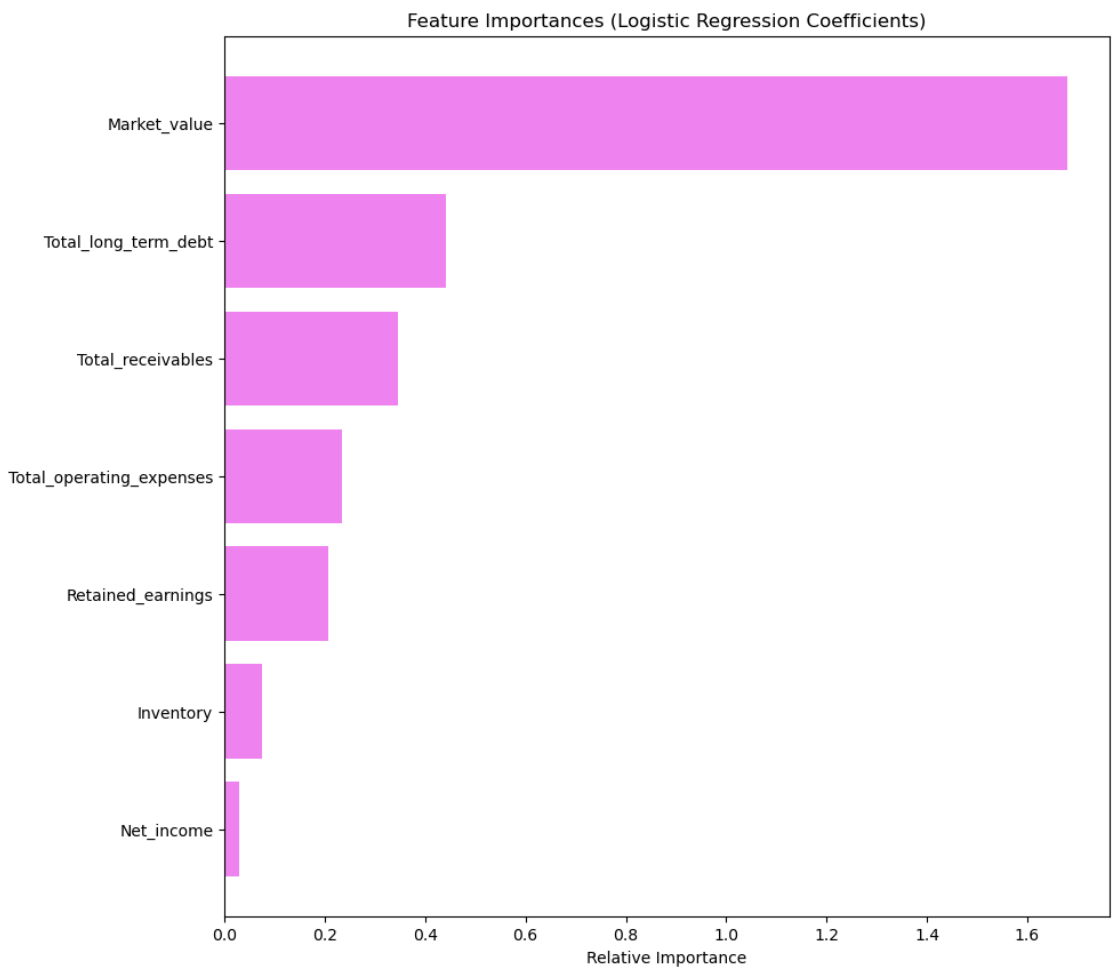
```
Testing performance comparison:
```

[87]:
```
           Logistic Regression  Tuned Logistic Regression  Random Forest  \
Accuracy              0.790323                   0.419355       0.784274
Recall                0.019231                   0.673077       0.038462
Precision             0.500000                   0.216049       0.363636
F1                    0.037037                   0.327103       0.069565

           Tuned Random Forest
Accuracy              0.653226
Recall                0.384615
Precision             0.270270
F1                    0.317460
```

- The Tuned Random Forest is the best option, as it strikes a balance between precision, recall, and F1 score. Unlike the untuned version, it avoids overfitting while maintaining acceptable accuracy. ##### Justification for Selection:
- Balanced trade-off in performance metrics (Accuracy: ~65.32%, Recall: ~38.46%, F1 Score: ~31.75%) when compared to overfitting in the base Random Forest and poor recall in Logistic Regression models.
- Effective handling of feature interactions and complex data structures.

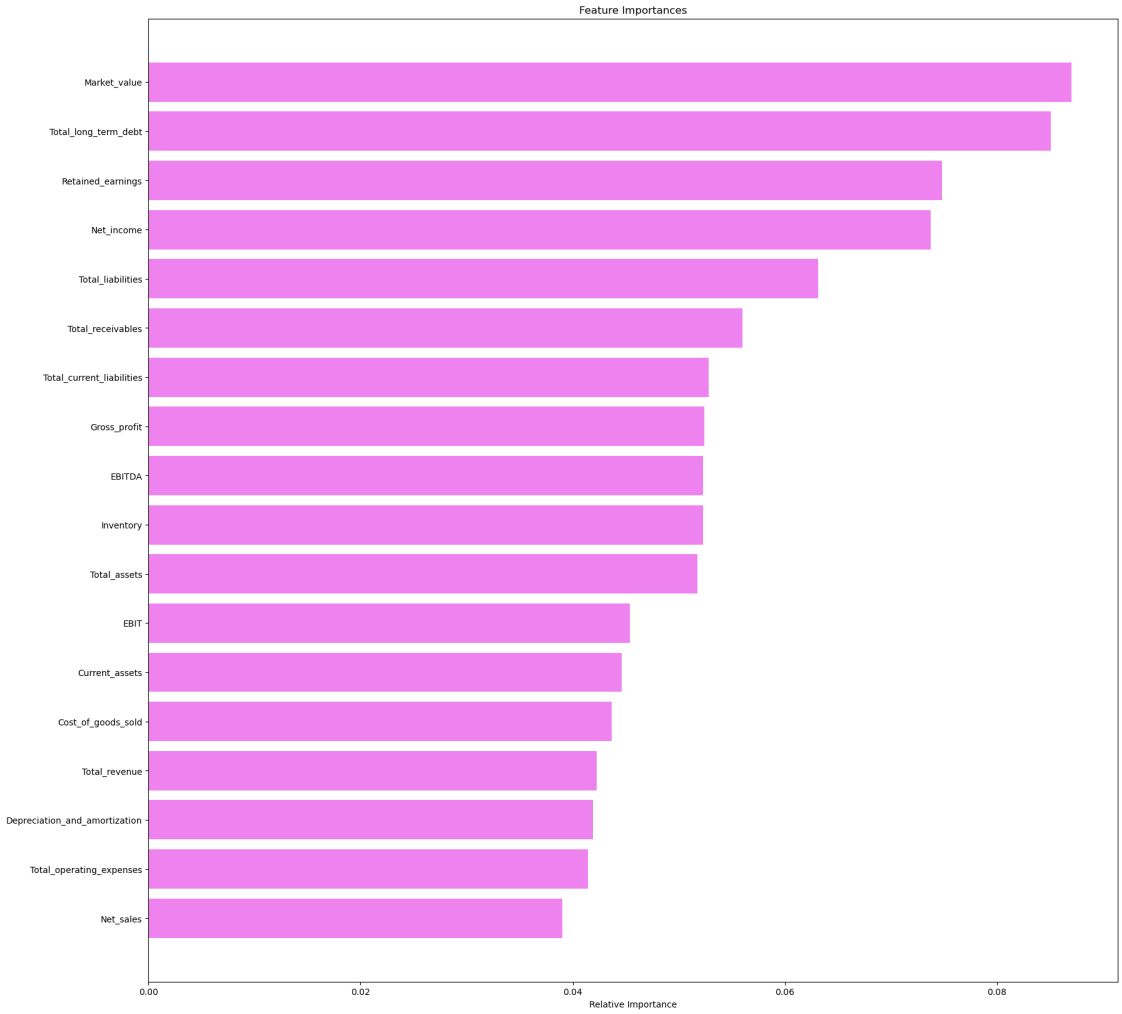
10.1 Feature Importance for Best Model

[88]:
```
# Uncomment and run to check feature importance for Improved Logistic␣
↪Regression model

coefficients = LogisticReg_improved.params.values[1:]  # Exclude the constant␣
↪term
relative_importance = np.abs(coefficients)
feature_names = X_train_new_with_intercept.columns[1:]
indices = np.argsort(relative_importance)

plt.figure(figsize=(10, 10))
plt.title("Feature Importances (Logistic Regression Coefficients)")
plt.barh(range(len(indices)), relative_importance[indices], color="violet",␣
↪align="center")
plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
plt.xlabel("Relative Importance")
plt.show()
```



- The feature Market_value has the highest relative importance, indicating it is the most influential variable in predicting bankruptcy.
- Total_long_term_debt: Companies with higher long-term debt are more likely to go bankrupt.
- Total_receivables: Higher receivables might indicate better financial health, reducing the risk of bankruptcy.
- Total_operating_expenses: Operating expenses have moderate importance, potentially indicating that excessive expenses might increase bankruptcy risk.
- Retained_earnings, Inventory, and Net_income show relatively lower importance compared to the top features.

[89]:
```
# Uncomment and run to check feature importance for Improved Random Forest model

feature_names = X_train.columns
importances = best_rf_classifier.feature_importances_
indices = np.argsort(importances)

plt.figure(figsize=(20, 20))
plt.title("Feature Importances")
plt.barh(range(len(indices)), importances[indices], color="violet",␣
 ↪align="center")
plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
plt.xlabel("Relative Importance")
plt.show()
```



- Market_value- emphasizing its strong predictive power for the target variable (likely Bankrupt). It aligns with expectations as a higher market value often indicates better financial health.
- Total_long_term_debt and Retained_earnings- reflecting their significant roles in assessing a company's financial stability. High long-term debt may increase bankruptcy risk, while retained earnings are a marker of profitability.
- Features like Net_income, Total_liabilities, and Total_receivables have moderate importance.
- Features like Depreciation_and_amortization, Total_operating_expenses, and Net_sales have lower importance in predicting bankruptcy.

10.1.1 The most critical features influencing bankruptcy predictions are:

- Total Liabilities: High liabilities are indicative of financial instability.
- Market_value: Indicates better financial health.
- Total Assets: A company with significant assets may have better leverage in tough times.
- Total Receivables: A reflection of cash inflows and credit risks.

- Total Revenue: Higher revenues generally correlate with a healthier financial position.

Focusing on these features can help stakeholders proactively identify and mitigate bankruptcy risks.

11 Conclusions and Recommendations

11.0.1 Conclusion

- The Tuned Random Forest model performed best, striking a balance between accuracy, recall, and precision. While its overall accuracy is 65.32%, it offers the best ability to identify bankruptcy cases (recall: 38.46%) compared to other models.
- Logistic Regression models struggled with recall, making them less suitable for this classification problem where identifying bankrupt companies is critical. ##### Key Predictors of Bankruptcy:
- Total Liabilities emerged as the most influential predictor, highlighting the importance of a company's debt load.
- EBITDA and Total Assets are critical indicators of operational efficiency and financial stability.
- Total Receivables reflect credit management, and Total Revenue highlights overall business performance. ### Recommendations: ##### Financial Monitoring:
- Focus on reducing Total Liabilities and maintaining a healthy EBITDA to mitigate bankruptcy risks.
- Monitor accounts receivables closely, ensuring efficient credit collection processes to avoid liquidity issues. ##### Adopt Predictive Tools:
- Use the Tuned Random Forest model to proactively assess the bankruptcy risk of companies, particularly for stakeholders in financial institutions, investors, and auditors. ##### Policy Recommendations for Companies:
- Companies at risk should focus on: Debt restructuring to lower liabilities. Cost optimization to improve EBITDA.

[ ]:

```
```